

案例结构

ATP 的案例存在关系型数据库里，不是脚本文件。理解字段结构有助于理解平台的能力边界——案例能表达什么、不能表达什么，本质上由这张表决定。

为什么是关系型结构

与脚本方案的取舍

很多自动化平台把案例存成 YAML 或直接存代码。ATP 选择关系型结构，是因为使用者是**不写代码的测试工程师**。

这个选择的代价和收益都很明确：

关系型结构 (ATP) 脚本 / YAML
--- --- ---
上手门槛 低，填表即可 要懂语法
表达能力 受限于是预定义 action 任意逻辑
批量修改 一条 SQL 搞定 要写脚本或手改
版本对比 字段级 diff 文本 diff
复杂分支 **做不到** 容易

所以 ATP 不适合表达复杂的条件分支和循环。遇到这类场景，正确做法是拆成多条案例，而不是想办法在一条案例里塞进逻辑。

案例主表 tc_case

标识与归属字段

字段 类型 说明
--- --- ---
`case_id` VARCHAR(32) 主键，平台生成的雪花 ID，用户不可见也不需要关心
`case_code` VARCHAR(64) **业务编号**，唯一，格式 `ATP-{MODULE}-{4位序号}`，这才是人和人之间沟通用的编号
`title` VARCHAR(200) 案例标题，必填
`module_id` VARCHAR(32) 所属模块，外键指向模块字典表，必填
`author` VARCHAR(64) 创建人，必填

`case\_id` 和 `case\_code` 的区分容易混淆。简单记：**跟人说话用 case_code，跟系统打交道用 case_id**。缺陷单里引用案例应该写 case_code。

执行控制字段

字段	类型	默认	说明
---	---	---	---
`priority`	ENUM	无，必填	`P0` / `P1` / `P2` / `P3`
`status`	ENUM	`DRAFT`	`DRAFT` / `ACTIVE` / `DEPRECATED`
`browser`	ENUM	`CHROME`	`CHROME` / `FIREFOX` / `EDGE`
`timeout_sec`	INT	30	整条案例的超时上限，范围 5~300
`precondition`	TEXT	空	前置条件，自然语言描述

关于 `timeout\_sec`：这是**整条案例**的上限，不是单个步骤的。单步骤等待时间由步骤自己的 `wait\_timeout\_sec` 控制。两者的关系是新人最常搞混的一点，详见《等待策略》。

如果一条案例需要超过 300 秒，说明它测的东西太多了，应该拆分。

步骤表 tc_step

定位与操作字段

字段	类型	说明
---	---	---
`step_id`	VARCHAR(32)	主键
`case_id`	VARCHAR(32)	外键，指向所属案例
`seq`	INT	执行顺序，**从 1 开始且连续，不允许跳号**
`action`	ENUM	动作类型，见《Action 参考》
`locator_type`	ENUM	`XPATH` / `CSS` / `ID` / `NAME` / `LINK_TEXT`，可空
`locator_value`	VARCHAR(512)	定位表达式，可空
`input_data`	VARCHAR(1024)	输入数据，可空
`expected`	VARCHAR(1024)	期望值，仅断言类 action 用
`description`	VARCHAR(500)	步骤说明，可空但建议填

`seq` 不允许跳号这一点由平台强制。在中间插入步骤时，平台会自动把后续步骤的 seq 整体后移，不需要手工调整。

等待与容错字段

字段	类型	默认	说明
---	---	---	---
`wait_strategy`	ENUM	无，必填	`NONE` / `PRESENCE` / `VISIBLE` / `CLICKABLE`
`wait_timeout_sec`	INT	10	该步骤的等待上限，范围 1~120
`on_failure`	ENUM	`ABORT`	`ABORT` / `CONTINUE` / `RETRY`

`on\_failure` 的三个取值语义：

- `ABORT` — 该步骤失败则整条案例立即终止，后续步骤不执行。****默认值，绝大多数情况用这个****
- `CONTINUE` — 记录失败但继续往下跑。只在「这一步失败不影响后续验证」时用，比如关闭一个可能不存在的引导弹窗
- `RETRY` — 失败后重试，最多 2 次。适合已知会偶发抖动的步骤

`RETRY` 容易被滥用。如果一个步骤需要靠重试才能稳定通过，****先怀疑等待策略写错了****，而不是加重试。重试会掩盖真正的问题，还会让案例执行时间变长。

模块字典 tc_module

模块字典是只读表，固定 8 个模块，不能自行增删：

module_id	module_code	名称
M001	LOGIN	ログイン / 登录认证
M002	SEARCH	検索 / 商品搜索
M003	CART	カート / 购物车
M004	ORDER	注文 / 订单管理
M005	USER	ユーザー管理 / 用户中心
M006	PAYMENT	決済 / 支付
M007	REPORT	レポート / 报表导出
M008	ADMIN	管理画面 / 后台管理

`module\_code` 会作为 `case\_code` 的中间段，所以模块一旦选定，改起来要连 `case\_code` 一起改。新建案例时想清楚归属，能省后面的事。模块边界的判断规则见《模块划分说明》。

案例之间没有依赖关系

ATP ****不支持案例之间声明依赖****。没有「案例 A 执行成功后才执行案例 B」这种编排能力。

这是有意为之。允许案例间依赖会导致两个后果：一是失败会级联，一条挂掉一片报错，定位成本很高；二是案例无法单独重跑，排查时非常痛苦。

需要共享前置状态时，正确做法是把前置操作****写进每条案例自己的步骤里****，或者用数据准备接口在 `precondition` 里说明。冗余几个步骤，换来的是每条案例都能独立执行、独立重跑。